Kazakh-British Technical University

Data collection and preparation

STUDENTS' INDEPENDENT STUDY (SIS) №1

Statistical Analysis of Persona Game Reviews

Full Name: Zhamshid

Kabanbay, Rassul Tampekov

ID: 23B031297, 23B0331449

28.10.2026

Almaty

Introduction

This project focuses on the statistical analysis of user reviews of Persona video games collected from two platforms: Steam and GameFAQs.

The main objective is to examine review characteristics, particularly review length, and investigate whether review length is associated with user ratings or recommendations.

The analysis covers the following aspects:

- **Review length distribution and descriptive statistics.**
- **Differences in review length between positively and negatively recommended Steam reviews.**
- **Correlation between review length and user ratings on GameFAQs.**
- **Comparison of review length between Steam and GameFAQs.**
- **Comparison of review statistics across individual games.**

Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and SciPy are used for data processing, visualization, and statistical testing.

Steam Review Collection

Steam reviews were collected using a Python script that sends HTTP requests to the Steam review API.

The collection process uses the following components:

- **Requests: Sends HTTP requests to retrieve review data.**
- **Pagination: Uses a cursor to retrieve reviews across multiple API responses.**
- **Review limits: Controls the maximum number of reviews requested for each game.**
- **Duplicate prevention: Tracks previously encountered review identifiers to avoid collecting the same review multiple times.**
- **Response validation: Checks HTTP responses and processes the returned JSON data.**

The collected reviews provide information about review text, recommendation status, and the corresponding game.

This approach enables automated data collection while reducing the risk of duplicate records.

```python
def get_steam_reviews(appid, n=100):
    url = "https://api.steampowered.com/IUserReviewsService/GetAppReviews/v1/"

    headers = {
        "User-Agent": "student-project/1.0",
        "Accept": "application/json"
    }

    reviews = []
    seen_ids = set()
    seen_cursors = set()
    cursor = "*"

    while len(reviews) < n:
        params = {
            "input_json": json.dumps({
                "appid": appid,
                "num_per_page": min(100, n - len(reviews)),
                "cursor": cursor,
                "filter": "recent"
            })
        }

        response = requests.get(
            url,
            headers=headers,
            params=params,
            timeout=15
        )
        response.raise_for_status()

        data = response.json().get("response", {})
        batch = data.get("reviews", [])

        if not batch:
            break

        for item in batch:
            review_id = item.get("recommendationid")

            if review_id is not None:
                unique_id = str(review_id)
            else:
                unique_id = (
                    str(appid),
                    item.get("language", ""),
                    item.get("review", "").strip()
                )

            if unique_id in seen_ids:
                continue

            seen_ids.add(unique_id)

            review_text = re.sub(
                r"\s+",
                " ",
                item.get("review", "")
            ).strip()

            reviews.append({
                "appid": appid,
                "review": review_text,
                "language": item.get("language", ""),
                "voted_up": item.get("voted_up")
            })

            if len(reviews) >= n:
                break

        next_cursor = data.get("cursor")
```

```
        if not next_cursor or next_cursor == cursor:
            break

        if next_cursor in seen_cursors:
            break

        seen_cursors.add(next_cursor)
        cursor = next_cursor

        time.sleep(0.2)

    return pd.DataFrame(
        reviews,
        columns=["appid", "review", "language", "voted_up"]
    )


appids = [1809700, 1113000, 2161700, 1687950]

all_reviews = []

for appid in appids:
    df = get_steam_reviews(appid, n=100)
    all_reviews.append(df)

reviews_df = pd.concat(all_reviews, ignore_index=True)

reviews_df = reviews_df.drop_duplicates(
    subset=["appid", "review", "language"]
).reset_index(drop=True)

reviews_df.to_csv(
    "steam_reviews_persona.csv",
    index=False,
    encoding="utf-8-sig"
)

print(reviews_df.shape)
print(reviews_df["voted_up"].value_counts())
```

```
(325, 4)
voted_up
True     304
False     21
Name: count, dtype: int64
```

GameFAQs Review Collection

GameFAQs reviews were collected using a Python script designed to extract structured information from review listing pages and individual review pages.

The script supports three collection modes:

- **Requests mode: Retrieves HTML pages using HTTP requests.**
- **Safari mode: Retrieves HTML through a dedicated Safari browser tab.**
- **Offline mode: Reuses previously saved HTML pages without requesting them again.**

The collection procedure includes:

- **Extracting review identifiers, authors, game titles, ratings, and review text.**
- **Recording publication and update dates.**
- **Validating ratings against the original 1–10 rating scale.**
- **Checking consistency between ratings shown on listing pages and individual review pages.**
- **Identifying review language using a character-based heuristic.**
- **Preventing duplicate review identifiers.**
- **Recording collection methods and timestamps.**

The script also saves intermediate results and collection coverage information, allowing the collected data to be validated and processed later.

Language identification is based on the dominant writing system and should therefore be treated as an approximation rather than a comprehensive language classification method.

GameFAQs Data Cleaning and Validation

After collection, the GameFAQs dataset was processed to identify duplicate cross-posts and validate data completeness.

The cleaning procedure includes:

1. **Collection coverage validation: Verifies that all eight selected games are represented and that collected review counts match the listing counts.**
2. **Review identifier validation: Checks that each collected review has a unique identifier.**
3. **Missing data validation: Checks whether review texts and publication dates are present.**
4. **Rating validation: Ensures that numerical ratings fall within the original 1–10 scale.**
5. **Cross-post detection: Identifies repeated reviews based on the same game, author, complete review text, and rating.**

When an exact cross-post is identified, the earliest published version is retained, while later copies are excluded from the cleaned dataset.

The original dataset is preserved separately, and excluded records are documented in an additional CSV file.

The script also generates summary statistics by game and a JSON validation report containing information about data completeness, duplicate handling, and collection methods.

This procedure improves dataset consistency and provides a reproducible record of the cleaning decisions.

```
1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings as wr
import os
os.chdir(os.getcwd())
print("Working directory:", os.getcwd())
steam = pd.read_csv("steam_reviews_persona.csv")
gamefaq = pd.read_csv("reviews_gamefaqs_clean.csv")
game_names = {
    1809700 : "persona_3_portable",
    1113000 : "persona_4_golden",
    2161700 : "persona_3_reload",
    1687950 : "persona_5_royal",
}
steam["game"] = steam["appid"].map(game_names)
steam["source"] = "steam"
steam = steam.drop(columns="appid")
gamefaq = gamefaq.rename(columns={"game_id":"game"})
gamefaq = gamefaq.rename(columns={"review_text":"review"})
gamefaq = gamefaq.drop(columns=["game_title", "review_id", "review_type_code", "author", "platform", "source_url", "listing_url", "listing_updated_date
gamefaq["language"] = gamefaq["language"].replace("en", "english")
```

Data Collection and Preprocessing

Two datasets were used in this analysis: Steam reviews and GameFAQs reviews.

The Steam dataset contains user reviews, recommendation indicators (voted_up), language information, and game identifiers. Game identifiers were mapped to game names to make the dataset easier to interpret.

The GameFAQs dataset contains review texts and numerical ratings. Column names were standardized to ensure compatibility with the Steam dataset. Unnecessary metadata columns were removed, while the original rating and maximum rating values were retained.

Both datasets were assigned a common structure, including the following variables:

- **review — review text.**
- **game — game identifier or name.**
- **source — platform from which the review was collected.**
- **language — review language.**

The datasets were prepared for further statistical analysis.

Data Quality Assessment

Before conducting the analysis, both datasets were examined for missing values, duplicate rows, and data types.

The Steam dataset contains no missing values or complete duplicate rows. Its variables have appropriate data types: review text and categorical variables are stored as objects, while the recommendation indicator is initially stored as a Boolean value.

The GameFAQs dataset also contains no missing values or complete duplicate rows. Review ratings are stored as floating-point numbers, while the maximum rating is stored as an integer.

These checks help ensure that the datasets are structurally consistent and ready for statistical analysis.

```
[2]: print("Missing values(Steam):")
display(steam.isna().sum())

print("Number of duplicate rows(Steam):")
print(steam.duplicated().sum())

print("Data types(Steam):")
display(steam.dtypes)

print("Missing values:")
display(gamefaq.isna().sum())

print("Number of duplicate rows:")
print(gamefaq.duplicated().sum())

print("Data types:")
display(gamefaq.dtypes)
```

```
Missing values(Steam):
review      0
language    0
voted_up    0
game        0
source      0
dtype: int64
Number of duplicate rows(Steam):
0
Data types(Steam):
review      object
language    object
voted_up      bool
game        object
source      object
dtype: object
Missing values:
game          0
source        0
rating        0
rating_max    0
review        0
language      0
dtype: int64
Number of duplicate rows:
0
Data types:
game          object
source        object
rating       float64
rating_max     int64
review        object
language      object
dtype: object
```

```
[3]: steam["review_len"] = steam["review"].str.len()
gamefaq["review_len"] = gamefaq["review"].str.len()
steam['word_count'] = steam['review'].str.split().str.len()
gamefaq['word_count'] = gamefaq['review'].str.split().str.len()
```

Review Length Calculation

To examine review length, two additional variables were calculated:

- **review_len — the number of characters in a review.**
- **word_count — the number of words in a review.**

Both measurements were calculated separately for the Steam and GameFAQs datasets using the review text.
The word_count variable is used as the primary measure of review length in the subsequent statistical analysis. It provides a more interpretable measure of how much text users write, regardless of differences in individual word lengths.

Correlation Analysis: GameFAQs

This section examines whether review length is associated with user ratings on GameFAQs.
The analysis uses three correlation methods:

- **Pearson correlation measures the strength and direction of a linear relationship.**
- **Spearman correlation measures the strength and direction of a monotonic relationship using ranked values.**
- **Kendall's Tau measures rank association and provides an additional non-parametric assessment.**

Before interpreting the results, the rating scale was checked. All 177 GameFAQs reviews use a maximum rating of 10, meaning that the original rating values are directly comparable.

```
[]: from scipy.stats import pearsonr, spearmanr, kendalltau, pointbiserialr
pearson = pearsonr(
    gamefaq['word_count'],
    gamefaq['rating']
)
spearman = spearmanr(
    gamefaq['word_count'],
    gamefaq['rating']
)
kendall = kendalltau(
    gamefaq['word_count'],
    gamefaq['rating']
)

print("Pearson:", pearson)
print("Spearman:", spearman)
print("Kendall:", kendall)
```

```
Pearson: PearsonRResult(statistic=np.float64(0.09965026177875748), pvalue=np.float64(0.1869504301884556))
Spearman: SignificanceResult(statistic=np.float64(0.07970700055114424), pvalue=np.float64(0.2916091253389889))
Kendall: SignificanceResult(statistic=np.float64(0.059057869209802466), pvalue=np.float64(0.2942772152600322 5))
```

Interpretation of Results

The correlation coefficients indicate a weak positive association between review length and user ratings.

- **Pearson correlation: r = 0.100, p = 0.187.**
- **Spearman correlation: ρ = 0.080, p = 0.292.**
- **Kendall correlation: τ = 0.059, p = 0.294.**

All three p-values are greater than the conventional significance level of 0.05.
Therefore, the analysis does not provide sufficient statistical evidence of a significant relationship between review length and user ratings on GameFAQs.
Although all three coefficients are positive, their magnitudes are very small. This suggests that review length is not meaningfully associated with the numerical rating in this sample.

Review Length and Recommendation on Steam

This section investigates whether the length of Steam reviews differs depending on whether users recommend a game.
Reviews are divided into two groups:

- **Positive reviews (voted_up = 1).**
- **Negative reviews (voted_up = 0).**

Descriptive statistics are calculated for both groups, including the number of observations, median review length, and interquartile range (IQR).
The Mann–Whitney U test is then applied to determine whether the distributions of review lengths differ statistically between the two groups.
This non-parametric test is appropriate for comparing two independent groups without assuming that the data follows a normal distribution.

```
]: from scipy.stats import mannwhitneyu


data = steam.dropna(subset=["voted_up", "word_count"])

positive = data[data["voted_up"] == 1]["word_count"]
negative = data[data["voted_up"] == 0]["word_count"]

print("Positive:", len(positive))
print("Negative:", len(negative))

print("\nPositive median:", positive.median())
print("Negative median:", negative.median())

print("\nPositive IQR:",
      positive.quantile(0.75) - positive.quantile(0.25))

print("Negative IQR:",
      negative.quantile(0.75) - negative.quantile(0.25))

print("\nMedian difference:",
      negative.median() - positive.median())

result = mannwhitneyu(
    negative,
    positive,
    alternative="two-sided",
    method="auto"
)

effect_size = (
    2 * result.statistic /
    (len(negative) * len(positive))
) - 1

print("\nMann-Whitney U:", result.statistic)
print("P-value:", result.pvalue)
print("Effect size:", effect_size)
```

```
Positive: 304
Negative: 21

Positive median: 37.5
Negative median: 121.0

Positive IQR: 75.25
Negative IQR: 347.0

Median difference: 83.5

Mann-Whitney U: 4317.5
P-value: 0.006899585357912206
Effect size: 0.3526002506265664
```

Interpretation of Results

The dataset contains 304 positive reviews and 21 negative reviews.

The median review length is:

- **Positive reviews: 37.5 words.**
- **Negative reviews: 121 words.**

The median difference is 83.5 words, indicating that negative reviews tend to be substantially longer in this sample.

The interquartile ranges are also different:

- **Positive reviews: 75.25 words.**
- **Negative reviews: 347 words.**

The Mann–Whitney U test produced a p-value of approximately 0.0069, which is below the 0.05 significance threshold. Therefore, there is statistically significant evidence of a difference in review length distributions between positive and negative Steam reviews.

The effect size is approximately 0.353, indicating a moderate difference in the tendency of observations between the two groups.

It is important to note that the negative group contains considerably fewer observations than the positive group. This imbalance should be considered when interpreting the results.

Correlation Analysis: Steam

In addition to comparing the two recommendation groups, correlation analysis was performed to examine the relationship between review length and recommendation status.

Since voted_up is a binary variable and word_count is continuous, the point-biserial correlation coefficient was used. Spearman correlation was also calculated to evaluate the rank-based association between the two variables.

The recommendation variable was converted from Boolean values to integers (0 and 1) to facilitate statistical calculations.

[6]:
```
steam["voted_up"] = steam["voted_up"].astype(int)
result = pointbiserialr(
    steam['voted_up'],
    steam['word_count']
)

print("Correlation:", result.statistic)
print("P-value:", result.pvalue)

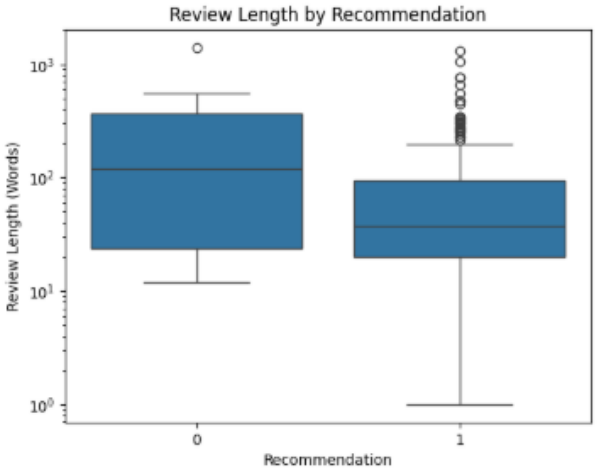
result_spearman = spearmanr(
    steam['voted_up'],
    steam['word_count']
)

print("Spearman:", result_spearman)
print(steam["voted_up"].value_counts())
```

```
Correlation: -0.2488312993981348
P-value: 1.1358805930585617e-05
Spearman: SignificanceResult(statistic=np.float64(-0.1501585071157076), pvalue=np.float64(0.006688186316871162))
voted_up
1    384
0     21
Name: count, dtype: int64
```

Interpretation of Results
The point-biserial correlation coefficient is approximately -0.241, with a p-value of 0.000011.
This indicates a statistically significant negative association between recommendation status and review length.
The Spearman correlation is also negative (ρ = -0.150, p = 0.0067), supporting the same general pattern.
Since positive recommendations are coded as 1 and negative recommendations as 0, the negative correlation indicates that longer reviews tend to be associated with negative recommendations.
These findings are consistent with the results of the Mann–Whitney U test, which also showed that negative reviews tend to be longer than positive reviews.
However, correlation does not imply causation. The results do not establish that review length causes users to recommend or reject a game.
Visualization 1: Review Length by Recommendation
A boxplot is used to compare the distribution of review lengths between positive and negative Steam reviews.
The vertical axis represents the number of words in each review. A logarithmic scale is applied because review lengths vary considerably, and a small number of very long reviews would otherwise make the differences between the groups difficult to observe.
The boxplot allows comparison of the median, interquartile range, and potential outliers in both groups.

7]:
```
sns.boxplot(steam, x = steam["voted_up"], y = steam["word_count"])
plt.xlabel("Recommendation")
plt.ylabel("Review Length (Words)")
plt.title("Review Length by Recommendation")
plt.yscale('log')
```

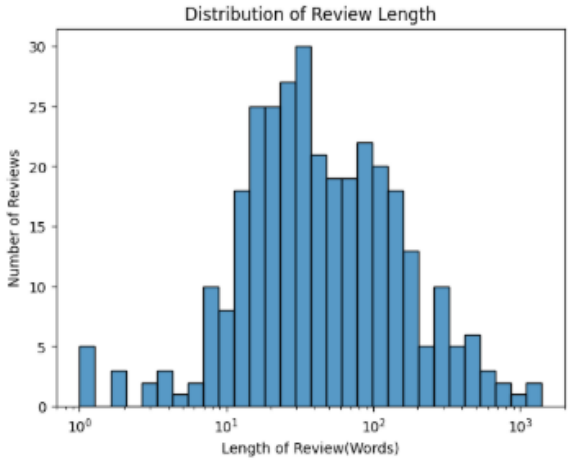


Interpretation
The boxplot illustrates that negative reviews generally have a higher median length and a wider spread than positive reviews.
Positive reviews are concentrated around shorter text lengths, while negative reviews show substantially greater variability.
This visual pattern supports the findings of the Mann–Whitney U test and correlation analysis.
The logarithmic scale makes it easier to observe both shorter reviews and unusually long observations.

Visualization 2: Distribution of Review Length
A histogram is used to examine the distribution of Steam review lengths.
The number of words is represented on the horizontal axis, while the vertical axis shows the number of reviews within each interval.
A logarithmic scale is applied to the horizontal axis to make the distribution easier to examine across a wide range of review lengths.

```
[8]: sns.histplot(steam, x = "word_count", bins = 30, log_scale = True)
     plt.xlabel("Length of Review(Words)")
     plt.title("Distribution of Review Length")
     plt.ylabel("Number of Reviews")
```

```
[8]: Text(0, 0.5, 'Number of Reviews')
```



Interpretation
The distribution of review lengths is strongly right-skewed.
Most Steam reviews are relatively short, while a smaller number of reviews contain substantially more words.
This indicates that the dataset includes a considerable variation in review length and that the mean may be influenced by unusually long reviews.
For this reason, the median and interquartile range are particularly useful descriptive statistics for this dataset.

Dataset Integration
The Steam and GameFAQs datasets were combined into a single DataFrame using concatenation.
A new variable, source, identifies the platform from which each review originated.
The combined dataset allows direct comparison of review characteristics across the two platforms.
After concatenation, additional quality checks were performed to identify missing values, duplicate rows, and changes in data types.

```
9]: alldata = pd.concat([steam, gamefaq], ignore_index=True)
    alldata.sample(5)
    print("Missing values:")
    display(alldata.isna().sum())

    print("Number of duplicate rows:")
    print(alldata.duplicated().sum())

    print("Data types:")
    display(alldata.dtypes)
```

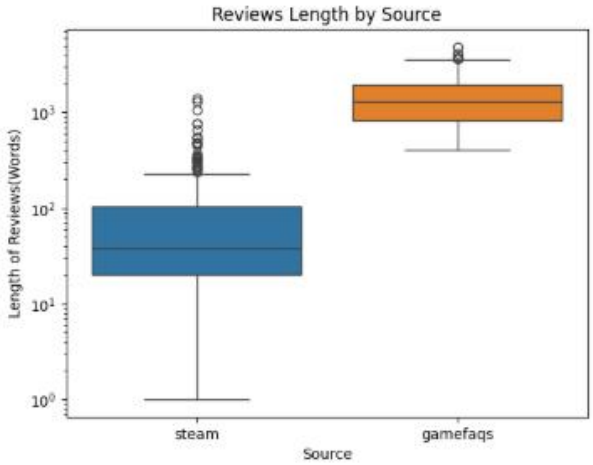
```
Missing values:
review          0
language        0
voted_up      177
game            0
source          0
review_len      0
word_count      0
rating        325
rating_max    325
dtype: int64
Number of duplicate rows:
0
Data types:
review         object
language       object
voted_up      float64
game           object
source         object
review_len      int64
word_count      int64
rating        float64
rating_max    float64
dtype: object
```

Interpretation of Data Quality Checks
The combined dataset contains 502 reviews: 325 from Steam and 177 from GameFAQs.
No complete duplicate rows were detected.
The missing values in voted_up, rating, and rating_max are expected because the two platforms provide different types of rating information.
Steam reviews contain recommendation indicators but do not provide the GameFAQs numerical rating fields.
Conversely, GameFAQs reviews contain numerical ratings but do not have the Steam recommendation indicator.
These missing values represent structural differences between the platforms rather than missing review text or failed data collection.

Visualization 3: Review Length by Source
A boxplot is used to compare review lengths between Steam and GameFAQs.
Both datasets use the same measurement, word_count, which allows comparison of the amount of text written by users on each platform.
A logarithmic scale is applied to account for the substantial difference in review lengths.

```
[10]: sns.boxplot(alldata, x = 'source', y = 'word_count', hue = 'source')
      plt.yscale('log')
      plt.title('Reviews Length by Source')
      plt.xlabel('Source')
      plt.ylabel('Length of Reviews(Words)')
```

```
[10]: Text(0, 0.5, 'Length of Reviews(Words)')
```



Interpretation
The visualization shows a substantial difference in review length between the two platforms.
GameFAQs reviews are considerably longer than Steam reviews. The median review length on GameFAQs is 1,291 words, compared with 38 words on Steam.
This difference suggests that the platforms encourage different review-writing practices.
However, the comparison should be interpreted carefully because the platforms may attract different types of users and provide different review formats.
Descriptive Statistics by Source
A summary table was created to compare the main characteristics of reviews from Steam and GameFAQs.
The following descriptive statistics were calculated:

- **Count — number of reviews.**
- **Mean — average number of words.**
- **Median — middle value of the distribution.**
- **Standard deviation — measure of variability.**
- **Q1 — first quartile (25th percentile).**
- **Q3 — third quartile (75th percentile).**

These statistics provide a more detailed comparison of review length across the two platforms.

```
[11]: summary_table = alldata.groupby('source').agg(
          count=('review', 'count'),
          mean_length=('word_count', 'mean'),
          median_length=('word_count', 'median'),
          std_length=('word_count', 'std'),
          Q1=('word_count', lambda x: x.quantile(0.25)),
          Q3=('word_count', lambda x: x.quantile(0.75))
      ).round(2).reset_index()

      display(summary_table)
```

	source	count	mean_length	median_length	std_length	Q1	Q3
0	gamefaqs	177	1472.54	1291.0	834.20	825.0	1941.0
1	steam	325	95.65	38.0	162.78	20.0	104.0

Interpretation
The summary statistics confirm the substantial difference in review length between the two platforms.
GameFAQs has a mean review length of approximately 1,472.54 words and a median of 1,291 words.
Steam reviews are much shorter, with a mean of approximately 95.65 words and a median of 38 words.
The difference between the mean and median on both platforms suggests right-skewed distributions, particularly for Steam.
The standard deviation is also considerably higher on GameFAQs in absolute terms, reflecting the wider range of review lengths.
Overall, GameFAQs reviews are substantially longer, while Steam reviews are generally shorter and more concise.

Descriptive Statistics by Individual Game

To examine differences between individual games, the dataset was grouped by both source and game.

The same descriptive statistics were calculated for each group: review count, mean, median, standard deviation, and quartiles.

This additional breakdown makes it possible to examine variation within each platform rather than relying exclusively on overall platform averages.

```
summary_table_game = alldata.groupby(['source', 'game']).agg(
    count=('review', 'count'),
    mean_length=('word_count', 'mean'),
    median_length=('word_count', 'median'),
    std_length=('word_count', 'std'),
    Q1=('word_count', lambda x: x.quantile(0.25)),
    Q3=('word_count', lambda x: x.quantile(0.75))
).round(2).reset_index()

display(summary_table_game)
```

	source	game	count	mean_length	median_length	std_length	Q1	Q3
0	gamefaqs	persona_3	33	1419.33	1265.0	688.25	845.00	2023.00
1	gamefaqs	persona_3_fes	23	1741.48	1510.0	1027.90	914.50	2338.50
2	gamefaqs	persona_3_portable	12	741.83	662.0	317.78	476.75	985.50
3	gamefaqs	persona_3_reload	3	1668.33	1227.0	1073.35	1056.50	2059.50
4	gamefaqs	persona_4	52	1414.63	1278.5	792.34	784.00	1888.50
5	gamefaqs	persona_4_golden	20	1810.75	1825.5	671.95	1384.00	2023.00
6	gamefaqs	persona_5	24	1302.46	992.5	732.57	888.25	1564.75
7	gamefaqs	persona_5_royal	10	1880.60	1697.0	1274.49	938.75	2267.25
8	steam	persona_3_portable	25	60.72	6.0	141.95	2.00	36.00
9	steam	persona_3_reload	100	126.93	50.0	227.82	23.75	121.50
10	steam	persona_4_golden	100	75.76	29.0	124.77	15.00	87.50
11	steam	persona_5_royal	100	92.99	46.5	112.51	25.75	121.00

Interpretation

The results show variation in review length across individual games on both platforms.

On GameFAQs, the median review length ranges from 662 words for Persona 3 Portable to 1,825.5 words for Persona 4 Golden.

On Steam, the median ranges from 6 words for Persona 3 Portable to 50 words for Persona 3 Reload.

The results suggest that review length varies not only between platforms but also between individual games.

However, the number of observations differs considerably across games. In particular, some GameFAQs groups contain relatively few reviews, which limits the reliability of direct comparisons.

Therefore, these differences should be interpreted descriptively rather than as evidence of statistically significant differences between individual games.

Conclusion

This analysis examined review length and its relationship with user ratings and recommendations across Steam and GameFAQs.

Several key findings emerged:

1. **Steam recommendation and review length: Negative Steam reviews tend to be longer than positive reviews. The Mann–Whitney U test confirmed a statistically significant difference between the two groups ($p = 0.0069$).**
2. **Steam correlation analysis: Both point-biserial and Spearman correlation coefficients showed a statistically significant negative association between recommendation status and review length.**
3. **GameFAQs correlation analysis: No statistically significant relationship was identified between review length and numerical ratings. Pearson, Spearman, and Kendall correlation coefficients were all weak and statistically insignificant.**
4. **Platform comparison: GameFAQs reviews are substantially longer than Steam reviews. The median review length is 1,291 words on GameFAQs and 38 words on Steam.**
5. **Differences between games: Review length varies across individual games, although unequal sample sizes limit the interpretation of these differences.**

Overall, the findings suggest that review length is associated with recommendation status on Steam, but not significantly associated with numerical ratings on GameFAQs.

Limitations

Several limitations should be considered when interpreting the results:

- **The datasets contain different numbers of reviews across platforms and individual games.**
- **The number of negative Steam reviews is considerably smaller than the number of positive reviews.**
- **Steam and GameFAQs use different review formats and rating systems.**
- **The analysis examines associations and differences but does not establish causal relationships.**
- **The findings describe the collected samples and should not automatically be generalized to all Persona players.**

Despite these limitations, the analysis provides insight into differences in user review behavior across platforms and explores the relationship between review length and user feedback.